

PQC 서명 크기가 B+tree 구조에 미치는 구조적 병목 분석 및 저장 전략

이승원*, 김의재*, 강희주*, 김민서*, 김하경*, 방지원*, 서화정†

한성대학교(대학원생, 학부생*, 교수†)

Structural Bottleneck Analysis and Storage Strategies for PQC Signature Sizes in B+tree Structures

Seung-Won Lee*, Ui-Jae Kim*, Hui-Ju Kang*, Min-Seo Kim*,
Ha-Gyeong Kim*, Ji-Won Bang* and Hwa-Jeong Seo†

Hansung University(Graduate student, Undergraduate student*,
Professor†)

요약

양자 내성 암호(PQC) 서명은 고전 전자서명 알고리즘 대비 큰 크기를 가지며, 이를 관계형 데이터베이스에 저장할 경우 고정 페이지 크기 기반의 B+tree 구조에 직접적인 영향을 미친다. 레코드 페이지로드가 페이지 임계값을 초과하면 오버플로 페이지 체인이 생성되어 랜덤 I/O가 증가하고, 대용량 레코드로 인한 낮은 팬아웃은 범위 조회 시 순차 읽기 비용을 선형적으로 증가시킨다. 본 논문은 이 두 병목이 독립적으로 작용함을 실험적으로 분리하기 위해 네 가지 저장 전략을 설계하고 대규모 벤치마크를 수행하였다. 페이지 크기 확장만으로 오버플로 체인을 제거하면 삽입 처리량이 최대 3.9배, 범위 조회 성능이 최대 60% 개선되었으며, 수직 파티셔닝과 대용량 페이지를 결합한 전략은 중간 크기 서명에서 범위 조회 지연을 최대 76%까지 감소시켰다. 반면 약 17 kB급 초대형 서명에서는 오버플로 제거 이후에도 팬아웃 병목이 잔존하여 범위 조회 성능 회복이 어려웠다. 본 연구는 두 가지 독립적 병목을 실험적으로 분리하고, 서명 크기와 시스템 우선순위에 따른 저장 전략 선택 기준을 제시한다.

keyword: 양자 내성 암호, SQLite, B+tree, 오버플로, 팬아웃, 저장 전략

1. 서론

양자 내성 암호인 PQC는 양자 컴퓨터의 연산 능력으로도 해독할 수 없도록 설계된 암호 기술이다. 미국 국립 표준 기술연구소인 NIST에서는 해시 기반 알고리즘인 SPHINCS+ 전자서명 알고리즘을 최종적으로 양자 내성 디지털 서명 표준으로 선정하였다[1]. 더 나아가 한국의 KpqC 연구단에서는 격자 기반 알고리즘인 HAETAE 전자서명 알고리즘과 MPC-in-the-Head 기반 알고리즘인 AIMer 전자서명 알고리즘을 국가 표준으로 등록하기 위해 최종 심사와 검증 중이다[2].

ECDSA, RSA와 같은 고전 전자서명 알고리즘의 서명 크기는 바이트(B) 수준이다. 하지만 PQC 전자서명 알고리즘의 서명 크기는 킬로바이트(kB) 수준으로, 고전 전자서명 알고리즘에 비해 서명 크기가 크다. 이는 임베디드 관계형 데이터베이스의 고정된 페이지 크기를 기반으로 동작하는 B+tree에 직접적인 구조적 영향을 미친다. 하나의 레코드가 실제로 저장할 수 있는 용량을 초과하면 오버플로 페이지 체인이 생성되어 팬아웃 감소와 트리 깊이 증가를 유발하고 상당한 I/O

성능 저하로 이어진다[3].

본 연구는 SQLite를 기반으로 PQC 서명 저장에 B+tree 구조와 성능에 어떤 영향을 미치는지 분석하고, 이를 완화하기 위한 네 가지 저장 전략을 설계/평가한다. 이를 바탕으로 서명 크기별 저장 전략 선택 기준을 제시한다.

본 논문의 주요 기여는 다음과 같다:

1. 9개 서명 알고리즘(Classical 3종, PQC 6종)과 4가지 저장 전략의 조합에 대해 오버플로 페이지 수와 리프 페이지 수, 트리 깊이 등 B+tree 구조 지표를 정량적으로 측정한다.
2. 오버플로와 팬아웃이라는 두 병목이 독립적으로 작용함을 실험적으로 분리, 정량화한다.

2. 배경

2.1 암호 알고리즘

본 연구는 보안 레벨 1 기준 9종의 알고리즘을 평

가한다. 고전 전자서명 알고리즘 3종은 서명이 모두 4 kB 이하이므로 어떤 전략에서도 오버플로가 발생하지 않는다. 이는 B+tree 구조 변화의 기준선을 제시한다. 반면 PQC 알고리즘의 서명 크기[4], [5], [6]는 1.4 kB~17 kB이므로 상당수가 4 kB 오버플로 임계값을 초과한다.

알고리즘	유형	서명 크기(B)	공개키 크기(B)
Ed25519	Classical	64	32
ECDSA-256	Classical	72	65
RSA-2048	Classical	256	294
HAETA-2	KpqC	1,463	992
AlMer-L1	KpqC	5,904	33
SPHINCS+	NIST	7,856	32
-SHAKE-128s	PQC		
SPHINCS+	NIST	7,856	32
-SHA2-128s	PQC		
SPHINCS+	NIST	17,088	32
-SHAKE-128f	PQC		
SPHINCS+	NIST	17,088	32
-SHA2-128f	PQC		

표 1. 평가 대상 알고리즘 비교 (보안 레벨 1)

2.2 SQLite B+tree와 오버플로

SQLite는 기본적으로 페이지 크기 4 kB를 사용한다. 레코드 페이지가 페이지 임계값을 초과하면 초과한 부분을 연결 리스트 형태의 오버플로 페이지 체인에 저장한다[3]. 이 구조는 대용량 BLOB 저장을 가능하게 하지만, 순차 조회 시 다수의 추가 랜덤 I/O를 강제적으로 발생시켜 성능을 저하시킨다[7].

3. 저장 전략 제안

PQC 서명이 B+tree 성능에 미치는 영향은 두 가지 구조적 요인에서 비롯된다. 첫째, 레코드 페이지가 페이지 임계값을 초과할 때 생성되는 오버플로 페이지 체인으로, 순차 조회 시 다수의 랜덤 I/O를 강제한다. 둘째, 대용량 레코드로 인해 페이지당 수용 레코드 수가 감소하는 낮은 팬아웃으로, 범위 조회 시 순차 읽기 비용이 선형 증가한다. 이하의 전략 설계와 실험은 두 요인을 독립적으로 제어하고 분리 측정하는 것을 목표로 한다.

페이지 크기 확장은 오버플로 임계값을 높여 체인 생성을 억제하고, 수직 파티셔닝은 서명 데이터를 별도 테이블로 분리하여 메타데이터 조회 시 불필요한 대용량 BLOB 읽기를 회피하는 것을 목적으로 한다. 단, 서명 데이터를 포함하는 범위 조회에서는 BLOB 테이블 JOIN이 불가피하므로 이 효과는 제한적이다.

(표 2)와 (표 3)과 같이 네 가지 전략을 설계한다. 전략 C는 오버플로를 제거하지 않은 채 수직 파티셔닝만 적용한 대조군으로, 테이블 분리의 독립적 효과를 측정하기 위해 포함하였다. C와 D의 비교를 통해

전략	페이지 크기	구조	오버플로 임계값
A	4 kB	단일 테이블	4,061 B
B	32 kB	단일 테이블	32,733 B
C	4 kB + 4 kB	meta + BLOB 테이블	4,061 B
D	4 kB + 64 kB	meta + BLOB 테이블	65,501 B

표 2. PQC 서명 저장 4가지 전략

전략	설계의도
A	기준선
B	오버플로 억제 효과 측정
C	수직 파티셔닝 단독 효과 분리(대조군)
D	오버플로 억제 + 팬아웃 향상 복합

표 3. PQC 서명 저장 4가지 전략 설계 의도

BLOB 페이지 크기 확장만의 효과를 추출할 수 있으나, 파일 분리에 따른 OS 수준 I/O 스케줄링 차이가 내재됨을 유의해야 한다.

4. 실험 환경 및 방법

실험은 Apple M2 (8GB RAM, 250GB NVMe SSD) 환경에서 SQLite 3.51.0 / Python 3.14.2를 사용하여 수행하였다. 암호화 연산 시간 배제를 위해 공개키, 서명, 메시지 해시를 미리 생성하여 서명 풀(10,000개)을 인덱스 순서로 순환 재사용하였다. SQLite 설정은 WAL 모드와 synchronous=NORMAL 옵션을 적용하였으며, 캐시 크기는 전략 간 유효 페이지 수를 동등한 조건으로 맞추기 위해 전략 A와 C는 64 MB, 전략 B는 512 MB, 전략 D는 64 MB로 설정하였다. 단, 전략 D의 BLOB DB(64 kB 페이지)는 ATTACH 구조상 별도 캐시 설정이 적용되지 않아 SQLite 기본값(약 2 MB)을 사용한다. 이는 전략 D의 범위 조회 성능이 실제보다 불리하게 측정될 수 있음을 의미한다.

벤치마크는 알고리즘·전략 조합당 500건 배치 단위로 100만 건 삽입 후 벌크 삽입 처리량, 범위 조회(10만 건) 지연을 측정하였다. 단, 고전 전자서명 알고리즘 3종은 모든 전략에서 오버플로가 이론적으로 발생하지 않으므로 전략 A만 측정하여 기준점으로 사용하였다.

오버플로 페이지 수, 리프 페이지 수, 트리 깊이와 같은 B+tree 구조 지표는 SQLite의 dbstat 가상 테이블로 측정하였다. 모든 벤치마크는 5회 반복 수행하였으며, 5회 전체의 평균을 최종 수치로 사용하였다.

5. 실험 결과

5.1 B+tree에서의 오버플로 발생 양상

(그림 1)과 같이 고전 전자서명 알고리즘과 HAETAE-2는 모든 전략에서 오버플로가 발생하지 않는다.

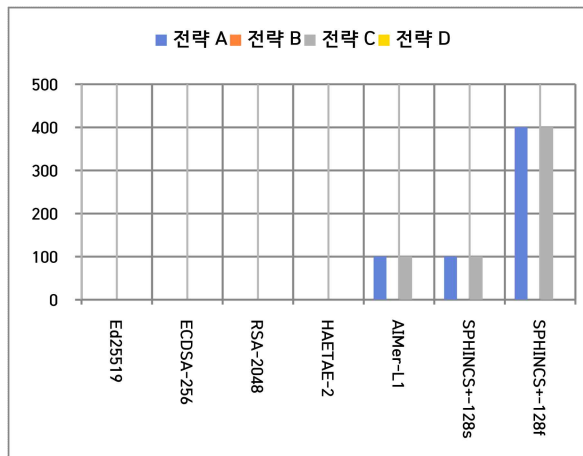


그림 2. 오버플로 페이지 수 (단위: 만 개)

AImer-L1과 SPHINCS+-128s는 전략 A와 C에서 각각 100만 개의 오버플로 페이지를 생성하고 전략 B와 D에서 완전히 제거된다. SPHINCS+-128f는 전략 A와 C에서 400만 개의 오버플로 페이지가 생성되며, 전략 B와 D에서는 오버플로가 제거된다. 즉 오버플로 제거 여부는 페이지 크기에 의해 결정되며, 전략 C만으로는 오버플로를 억제할 수 없다.

오버플로 제거 이후에도 서명 크기 자체가 팬아웃에 영향을 미친다. (그림 2)와 같이 HAETAE-2는 모든 전략에서 오버플로가 없음에도 전략 A에서 리프 페이지 수가 100만 개에 달한다. 이는 4 kB 페이지 공간을 레코드 1건이 거의 전부 점유하기 때문이다. AImer-L1은 전략 A에서 50만 개, 전략 B에서 20만 개로 감소하여 팬아웃이 향상된다. SPHINCS+-128f는 전략 B에서 리프 페이지 수가 100만 개로, 32 kB 페이지당 1건만 수용되는 극단적으로 낮은 팬아웃 상태이다. 반면 전략 D의 64 kB BLOB DB에서는 리프 페이지 수가 356,923개로 감소한다. 모든 PQC 알고리즘의 전략 A에서 100만 건 삽입 시 B+tree 길이는 4로 측정되었으나 전략 B·C·D에서는 3으로 감소한다. 이는 고전 전자서명 알고리즘의 전략 A 길이와 동일하다.

5.2 삽입 성능

전략 A에서 Ed25519는 14,173 rps(건/초), RSA-2048은 12,011 rps를 기록하였다. 이 값은 PQC 알고리즘의 성능 저하를 평가하는 기준점이다. 전략 A 대비 전략 B에서 오버플로 제거 후 AImer-L1은 5,981에서 23,400 rps로 3.9배, HAETAE-2는 12,107에서 27,706 rps로 2.3배, SPHINCS+-128s는 7,684에서 22,530 rps로 2.9배 향상되었다. 전략 C는 오버플로 여부와 무관하게 멀티 테이블 INSERT와 외래 키 검증 오버헤드로 인해 삽입 성능이 네 전략 중 최저를 기록하였다. HAETAE-2와 같이 오버플로가 없는 경우에도 동일한 결과가 나타나, 성능 저하 원인

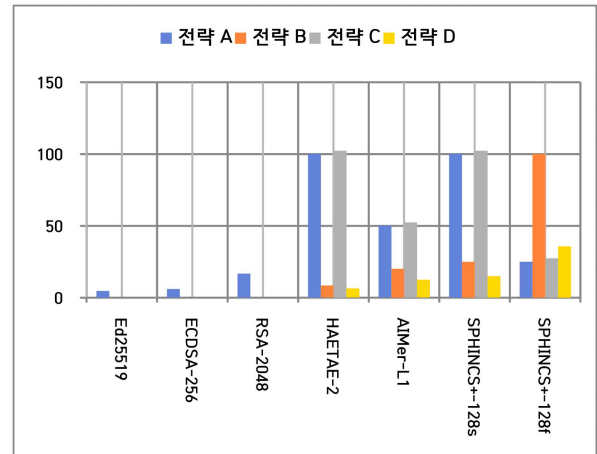


그림 1. 리프 페이지 수 (단위: 만 개)

이 오버플로가 아닌 테이블 분리 자체에 있음을 확인하였다. HAETAE-2는 전략 A 대비 51%, AImer-L1은 34% 감소하였다.

5.3 범위 조회 성능: 두 가지 독립적 병목

병목 1: 오버플로 페이지 체인에 의한 랜덤 I/O

전략 A에서 SPHINCS+-128f의 10만 건 범위 조회 지연은 최대 23,186ms로, Ed25519 대비 약 157~188배에 달한다. 전략 C는 오버플로를 제거하지 못한 채 JOIN 오버헤드까지 가중되어 지연이 전략 A보다 오히려 악화된다. 페이지 크기를 확장해 오버플로를 제거하면 이 병목은 해소되며, 전략 D에서 AImer-L1은 전략 A 대비 76%, HAETAE-2는 70%, SPHINCS+-128s는 68% 개선되었다. 반면 전략 B는 오버플로 제거만으로 AImer-L1의 범위 조회 지연이 58%, HAETAE-2가 60%에 그쳐, BLOB DB 64 kB 페이지로 팬아웃까지 향상시킨 전략 D가 추가적인 개선을 달성하였다.

병목 2: 낮은 페이지 팬아웃에 의한 대용량 순차 읽기

SPHINCS+-128f는 전략 B에서 오버플로가 완전히 제거되었음에도 범위 조회 지연이 전략 A와 유사하거나 오히려 소폭 증가하는 경우도 나타났다. 이는 32 kB 페이지 1건 단위 읽기로 인한 대용량 순차 I/O가 오버플로 제거 효과를 상쇄하기 때문이다. (그림 2)와 같이 리프 페이지 수가 100만 개이므로 10만 건 범위 조회 시 약 3.2GB를 순차적으로 읽어야 하는 구조적 한계가 그 원인이다. 전략 D에서도 21,905ms로 개선 효과가 나타나지 않아, 오버플로를 제거하고 파일까지 분리하였음에도 초대형 서명 자체가 야기하는 낮은 팬아웃이 근본 병목으로 작용함을 보여준다.

5.4 저장 공간

(표 4)는 전략별 총 저장 공간을 비교한다. 전략 B는 오버플로를 제거하는 대신 큰 페이지에 데이터를 인라인으로 저장하여 SPHINCS+-128f 기준 최대 1.9배의 공간 증가가 발생한다. HAETAE-2는 32 kB 페이지에서 팬아웃이 향상되어 저장 공간이 오히려 25% 감소한다. 전략 D는 전략 B 대비 저장 공간이 작아,

스토리지 제약 환경에서 현실적 대안이 된다.

알고리즘	전략 A	전략 B	전략 C	전략 D
HAETAE-2	3,916 MB	2,605 MB	4,007 MB	2,594 MB
AlMer-L1	5,864 MB	6,252 MB	5,955 MB	6,348 MB
SPHINCS+ -128s	7,822 MB	7,815 MB	7,913 MB	7,913 MB
SPHINCS+ -128f	16,604 MB	31,260 MB	16,695 MB	20,949 MB

표 4. 전략별 총 저장공간 (Table + Overflow 합산)

6. 저장 전략 선택 가이드라인

5절의 실험 결과를 바탕으로 서명 크기와 시스템 우선순위에 따른 전략 선택 기준을 (그림 4)와 같이 제시한다.

1. 소형 서명(HAETAE-2)은 모든 전략에서 오버플로가 발생하지 않아 삽입 처리량 우선 시 전략 B, 저장 공간과 범위 조회를 동시에 고려할 경우 전략 D가 적합하다.

2. 중간 크기 서명(AlMer-L1, SPHINCS+-128s)은 오버플로 체인이 지배적 병목이므로 전략 B가 삽입과 범위 조회 모두에서 가장 큰 개선을 보이며, 스토리지 제약 환경에서는 전략 D가 현실적 대안이다.

3. 초대형 서명(SPHINCS+-128f)은 어떤 전략으로도 범위 조회 성능 회복이 불가능한 구조적 한계가 존재한다.

전략 C는 오버플로를 억제하지 못한 채 테이블 분리 오버헤드만 가중되어 어떤 범주에서도 권장되지 않으며, 수직 파티셔닝의 효과는 페이지 크기 확장과 결합(전략 D)될 때에만 실현됨을 확인하였다.

7. 결론

PQC 서명이 SQLite B+tree 성능을 저하시킨다는 것은 예측 가능하지만, 본 연구는 그 원인이 오버플로와 팬아웃이라는 성격이 다른 두 병목의 결합임을 실험적으로 분리하였다는 점에서 의의가 있다.

페이지 크기 확장(전략 B)은 오버플로를 제거하여 삽입 처리량을 최대 3.9배, 범위 조회를 최대 60% 개선하지만, SPHINCS+-128f에서는 오버플로 제거 이후에도 범위 조회 지연이 전략 A와 유사하게 유지된다. 오버플로는 해소되었지만 팬아웃 병목이 독립적으로 작용하기 때문이다. 이는 두 병목을 구분하지 않으면 전략의 효과를 잘못 해석할 수 있음을 보여준다.

결국 서명 크기가 약 17 kB에 달하는 초대형 서명에서는 오버플로를 완전히 제거하고 파일까지 분리해도 팬아웃 자체가 근본 병목으로 남아, 현재 설계한 네 가지 전략 중 어느 것으로도 범위 조회 성능 회복

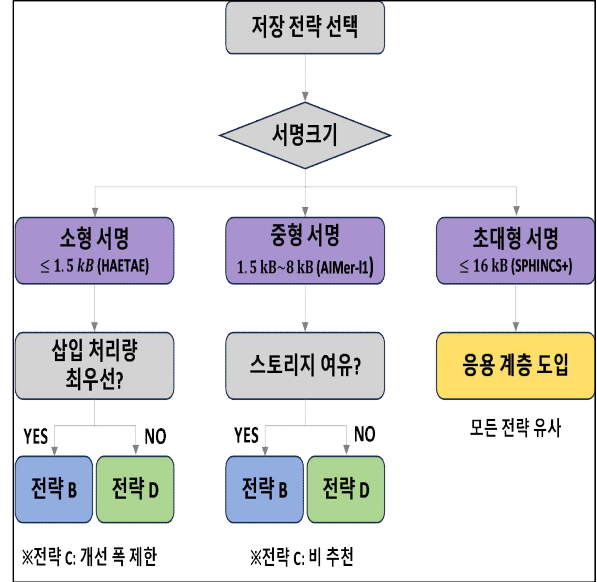


그림 4. 서명 크기 및 시스템 우선순위에 따른 저장 전략 선택 흐름

이 불가능한 구조적 한계가 존재한다. 이는 초대형 PQC 서명의 관계형 DB 저장이 페이지 기반 B+tree 구조 자체와 근본적으로 충돌함을 시사한다.

8. Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02306395, Development and Demonstration of PQC-Based Joint Certificate PKI Technology, 50%) and this work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-25394739, Development of Security Enhancement Technology for Industrial Control Systems Based on S/HBOM Supply Chain Protection, 50%).

[참고문헌]

- [1] National Institute of Standards and Technology (NIST), Stateless Hash-Based Digital Signature Standard, Federal Information Processing Standards Publications (FIPS) 205, August, 2024.
- [2] 양자내성암호연구단(Kpqc), 제1라운드 대상 알고리즘: HAETAE 및 AlMer, Kpqc 공모전, [Online] Available: https://www.kpqc.or.kr/contents/03_exhibit/sub_03.html, Accessed: March 26, 2026.
- [3] SQLite.org, Database File Format, [Online] Available :

<https://www.sqlite.org/fileformat.html>,
Accessed: March 26, 2026.

- [4] J. H. Cheon, et al., HAETAE: Shorter lattice-based fiat-shamir signatures, *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):25-75, 2024.
- [5] S. Kim, et al., AIM: Symmetric primitive for shorter signatures with stronger security (full version), *Cryptology ePrint Archive*, Paper 2022/1387, 2022.
- [6] D. J. Bernstein, et al., SPHINCS+, *Proc. ACM SIGSAC CCS '19*, pp. 2129-2146, Nov. 2019.
<https://doi.org/10.1145/3319535.3363229>.
- [7] M. Shin, Y. Jang, J. Lee, and S. Park, Analyzing Performance of Query Processing in SQLite on Android, *Proc. KCC 2014*, pp. 660-663, Apr. 2014.